

---

# Práctica 0

## Toma de contacto con Android Studio

*“¡Hola Mundo!”*

---

**Objetivo: Aprender a crear nuevos módulos en Android Studio y aprender a ejecutar tanto aplicaciones Java en escritorio como aplicaciones Android en emulador.**

### 1. Android y Java

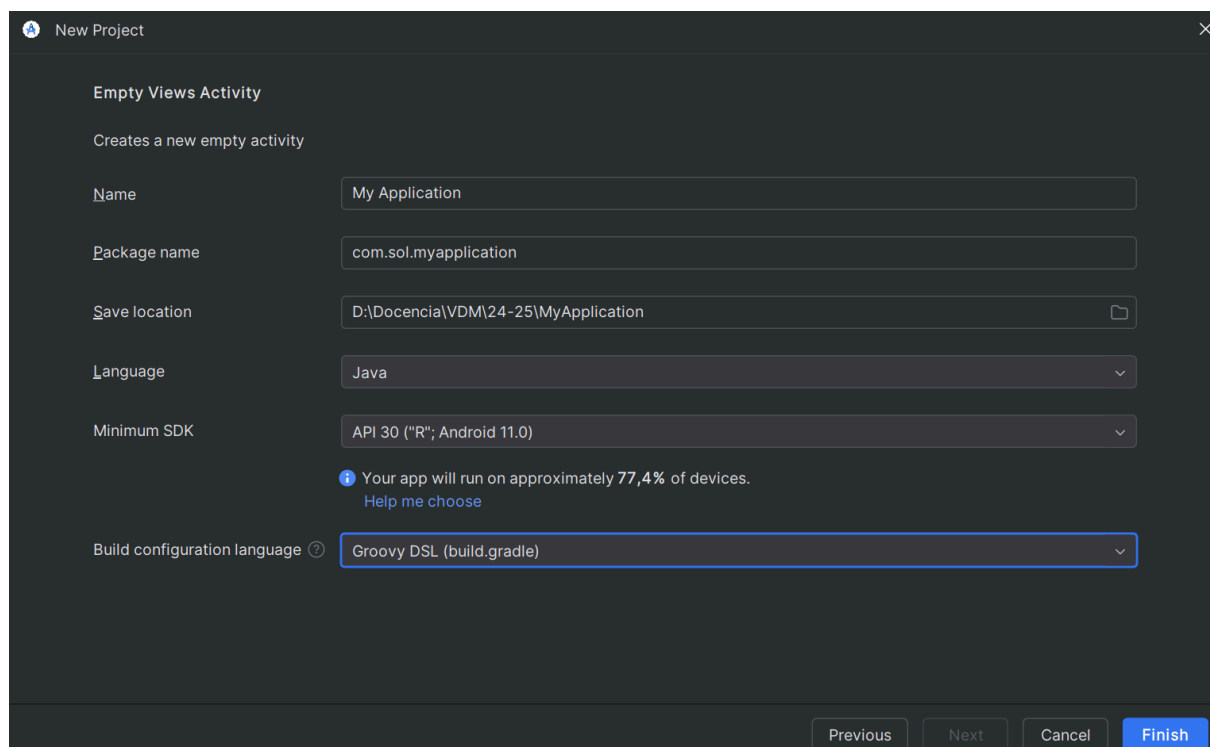
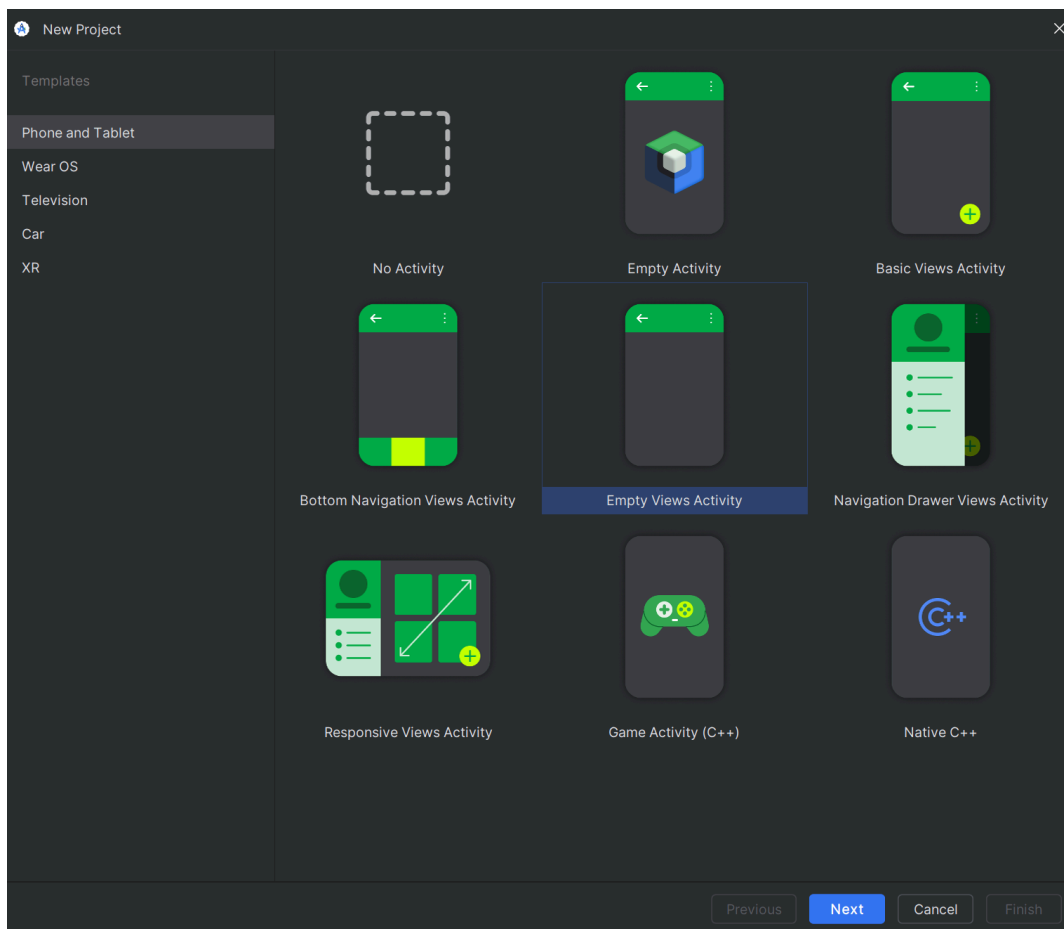
Android es un sistema operativo móvil basado en Linux y otros software de código abierto. Fue diseñado para dispositivos móviles con pantalla táctil, como teléfonos inteligentes, tabletas, relojes inteligentes, automóviles, televisores, etc. Inicialmente fue desarrollado por Android Inc., que fue adquirido por Google en 2005. Android fue presentado en 2007 junto con la fundación del Open Handset Alliance (un consorcio de compañías de hardware, software y telecomunicaciones) para avanzar en los estándares abiertos de los dispositivos móviles. El código fuente principal de Android se conoce como Android Open Source Project (AOSP), que se licencia principalmente bajo la licencia Apache. Tradicionalmente las aplicaciones para Android eran desarrolladas en lenguaje Java, si bien ahora también se pueden desarrollar usando Kotlin.

Java es un lenguaje de programación de alto nivel, basado en clases y orientado a objetos, diseñado para tener el menor número posible de dependencias de implementación. Java es un lenguaje compilado e interpretado. Las aplicaciones Java se compilan normalmente en bytecode que puede ejecutarse en cualquier máquina virtual Java (JVM) independientemente de la arquitectura del ordenador subyacente. El tiempo de ejecución de Java proporciona capacidades dinámicas (como la reflexión y la modificación del código en tiempo de ejecución) que normalmente no están disponibles en los lenguajes compilados tradicionales. La sintaxis de Java es similar a la de C y C++.

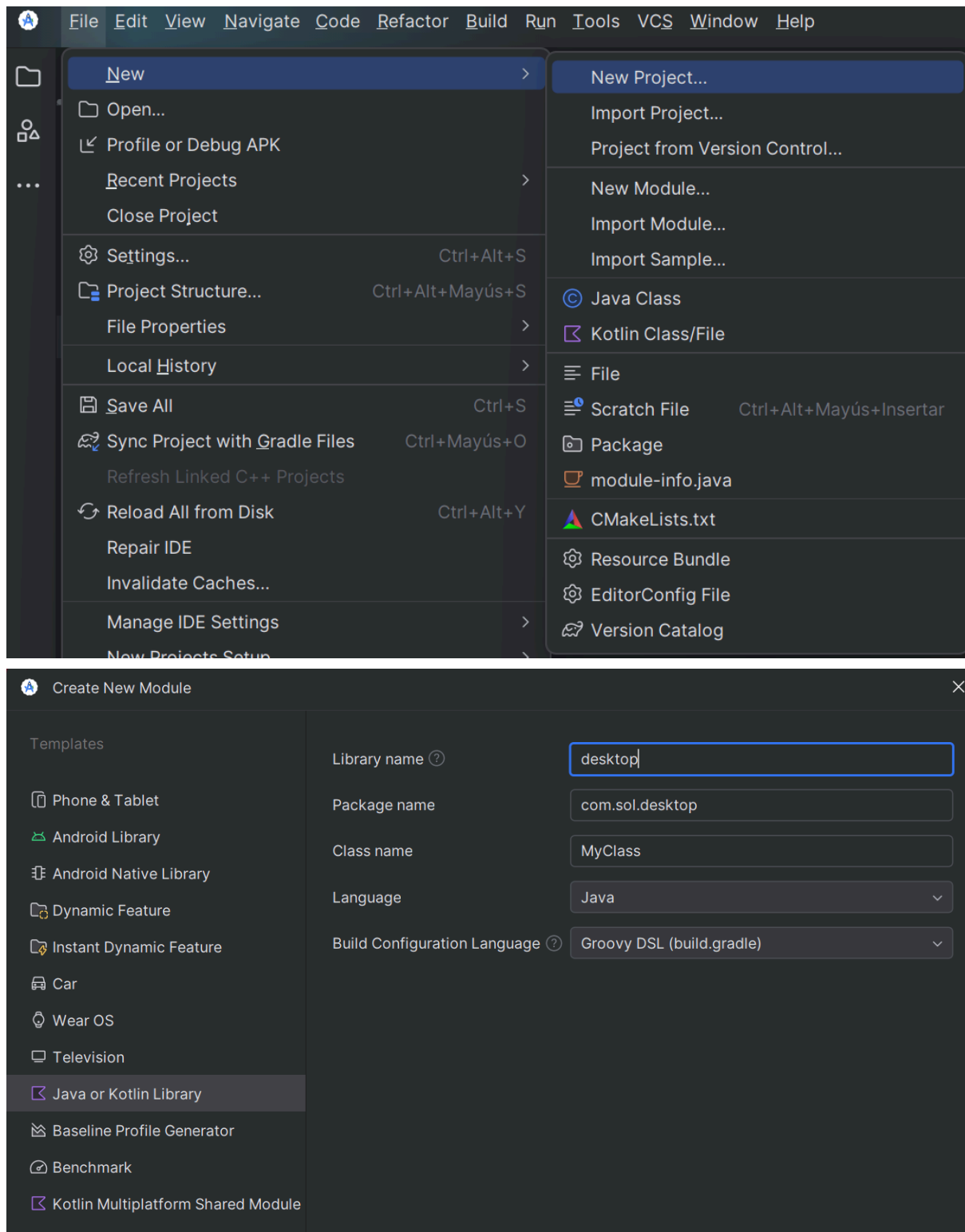
### 2. Descripción de lo que hay que desarrollar.

Durante esta práctica se deben desarrollar 2 aplicaciones muy básicas, de estilo ¡Hola Mundo!

Ambas aplicaciones deben estar contenidas en un mismo proyecto de Android Studio y debe poder ejecutarse una u otra. Para ello tendremos 2 módulos, uno de ellos de tipo **“Phone & Tablet”** con **“Empty Views Activity”** y el otro de tipo **“Java or Kotlin Library”**.



**Fig. 1.** Creación del proyecto.



**Fig. 2:** Creación de módulos

Al crear el proyecto ya tendremos un módulo “app” de tipo “*Phone & Tablet*”. Así que ahora tenemos que crear el resto de módulos que vayamos a necesitar. En este caso, de tipo Java.

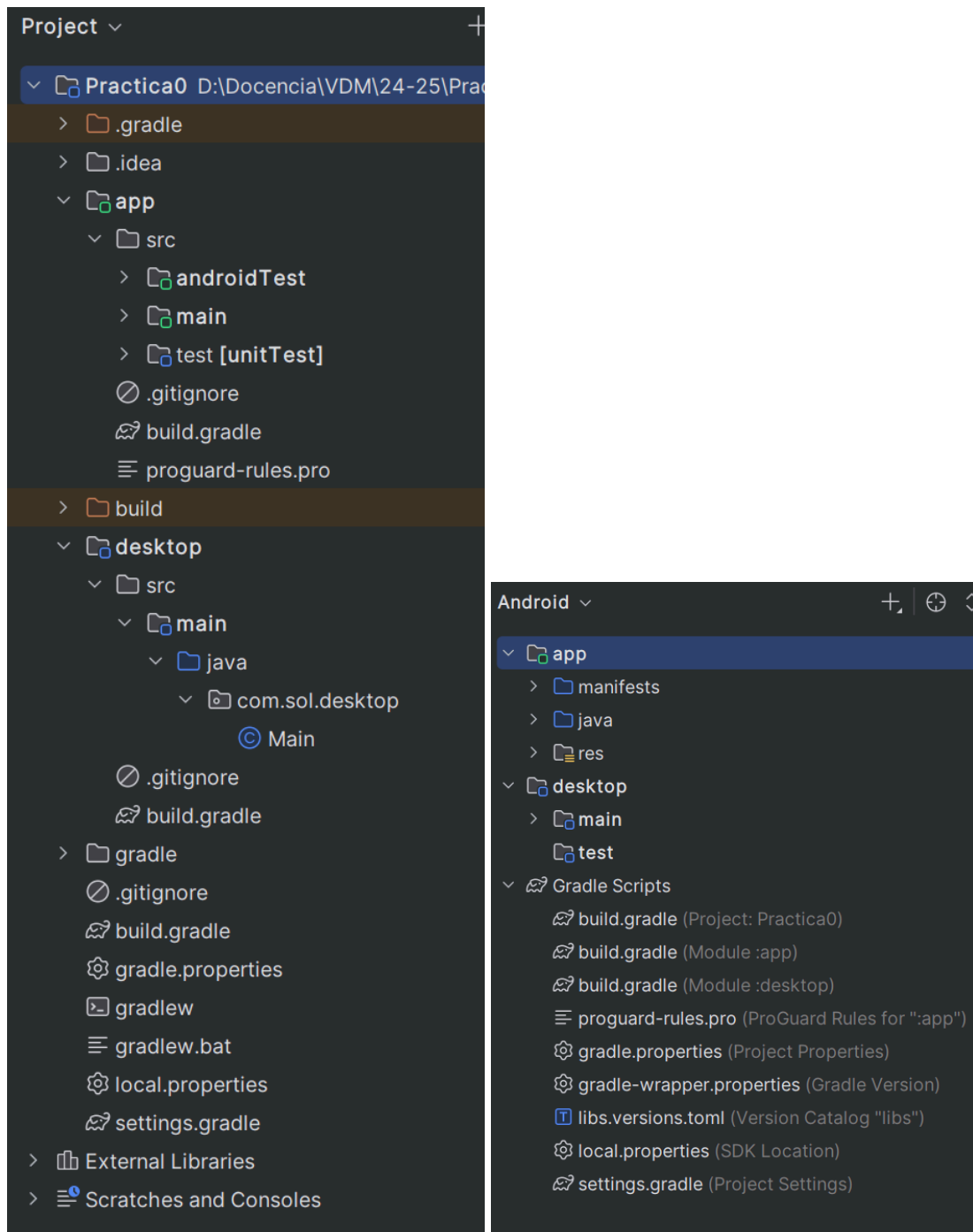


Fig. 3: Vista “project” (izquierda) y vista “Android” del proyecto

En el proyecto tendremos:

- **app > java > com.example.myfirstapp > MainActivity**  
Esta es la actividad principal. Es el punto de entrada de tu app. Cuando compilas y ejecutas la app, el sistema inicia una instancia del elemento [Activity](#) y carga su diseño.
- **app > res > layout > activity\_main.xml**  
Este archivo en formato XML define el diseño de la interfaz de usuario (IU) de la actividad. Contiene un elemento [TextView](#) con el texto "Hello, World!"

- **Gradle Scripts > build.gradle**

Hay tres archivos con este nombre: uno para el proyecto ("Project: My\_First\_App"), otro para el módulo de la app ("Module: My\_First\_App.app") y el último para el módulo de Java ("Module: My\_First\_App.desktop"). Cada módulo tiene su propio archivo `build.gradle`, pero este proyecto, por el momento, tiene un solo módulo. Usa el archivo `build.gradle` de cada módulo para controlar cómo el [complemento de Gradle](#) compila tu app. A fin de obtener más información sobre este archivo, puedes consultar [Cómo configurar tu compilación](#).

### 3. Un ¡Hola Mundo! en Android

Vamos a crear una primera aplicación muy básica en Android con el objetivo de familiarizarnos con la gestión de dependencias entre módulos, la creación de dispositivos virtuales y la depuración con Android Studio.

En esta aplicación tendremos dos vistas, la primera compuesta por un elemento que permite al usuario introducir texto y un botón. La segunda vista muestra únicamente el texto introducido en la primera vista después de pulsar sobre el botón.



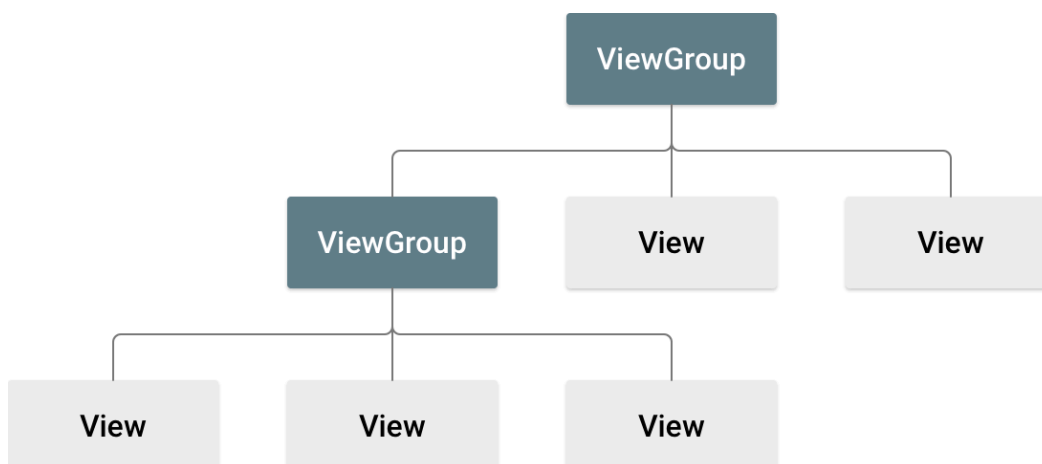
Fig. 4: Objetivo aplicación Android

**Nota:** Ten en cuenta que **más adelante**, nuestra interfaz estará compuesta por una única View, una [SurfaceView](#) que contendrá el canvas donde pintaremos nuestro juego. Y, si por ejemplo, quisiéramos tener un banner con anuncios, usaríamos otra vista más, un [AdView](#).

Para crear nuestra primera app de Android vamos a seguir los tutoriales que aparecen a continuación:

#### 1. Crear una app utilizando el editor de views de Android Studio

La interfaz de usuario (IU) de una app para Android se compila como una jerarquía de *diseños* y *widgets*. Los diseños son objetos [ViewGroup](#), contenedores que controlan cómo se posicionan sus vistas secundarias en la pantalla. Los widgets son objetos [View](#), componentes de la IU como botones y cuadros de texto.



**Figura 5:** Ilustración de cómo los objetos **ViewGroup** forman ramas en el diseño y contienen objetos **View**.

Android proporciona un vocabulario XML para las clases **ViewGroup** y **View**, por lo que la mayor parte de tu IU se define en archivos en formato XML. Sin embargo, en lugar de enseñarte a escribir en ese formato, en esta lección se te mostrará cómo crear un diseño con el editor de Android Studio. El editor de diseño escribe el archivo XML a medida que arrastras y sueltas vistas para compilar un diseño.

## Cómo abrir el editor de diseño

Para comenzar, configura tu lugar de trabajo de la siguiente manera:

1. En la ventana Android del proyecto, abre **app > res > layout > activity\_main.xml**.
2. A fin de hacer más espacio para el editor de diseño, oculta la ventana **Project**. Para ello, selecciona **View > Tool Windows > Project** o haz clic en **Project** a la izquierda de la pantalla de Android Studio.
3. Si el editor muestra la fuente XML, haz clic en la pestaña **Design**, en la parte

superior derecha de la ventana.



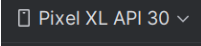
4. Haz clic en  (**Select Design Surface**) y selecciona **Blueprint**.

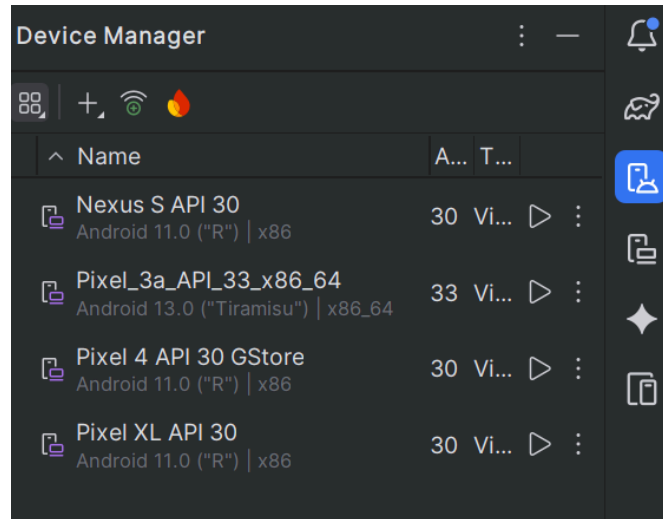


5. Haz clic en **(View Options)**, en la barra de herramientas del editor de diseño y asegúrate de que la opción **Show All Constraints** esté marcada.
6. Comprueba que la opción de conexión automática esté desactivada. La información

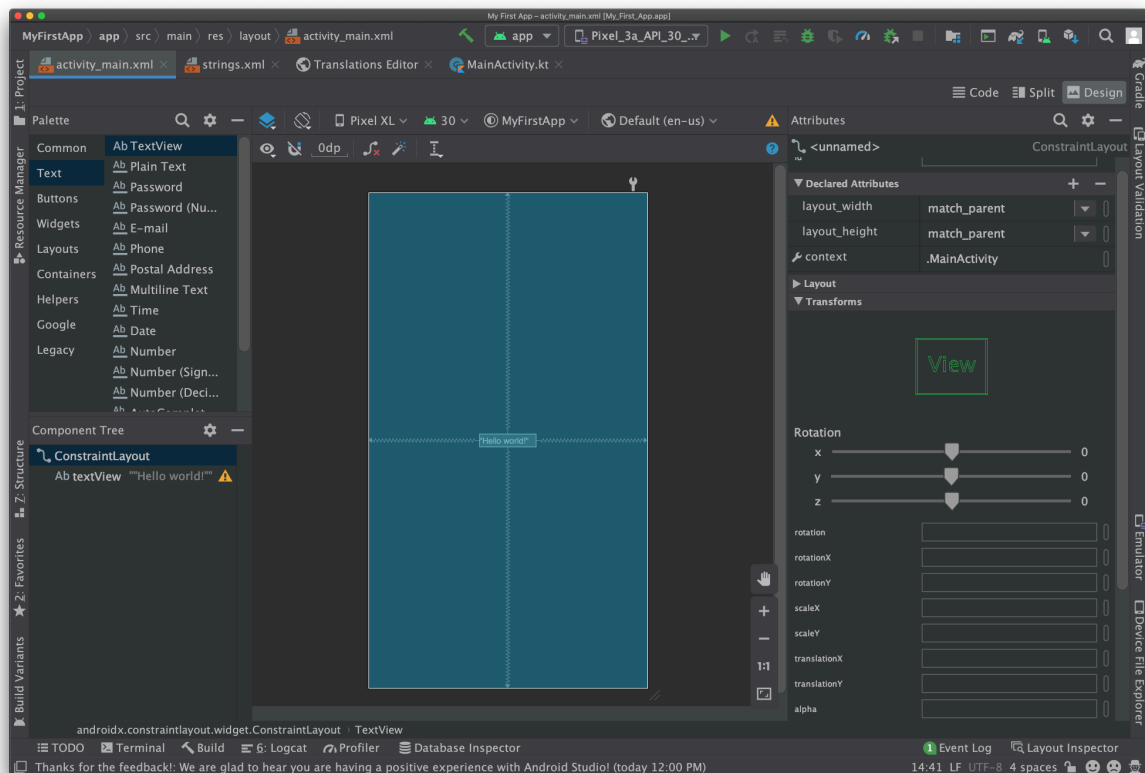
sobre la barra de herramientas muestra  (**Enable Autoconnection to Parent**) cuando la conexión automática está desactivada.

7. Haz clic en  (**Default Margins**) en la barra de herramientas y selecciona **16**. Si es necesario, puedes ajustar los márgenes para cada vista más adelante.

8. En la barra de herramientas, haz clic en  (Device for Preview) y selecciona **AVD: Pixel XL**. Es probable que tengas que crear el dispositivo virtual antes desde el menú de la derecha Device Manager:



El editor de diseño ahora se ve algo muy similar a la figura 6.

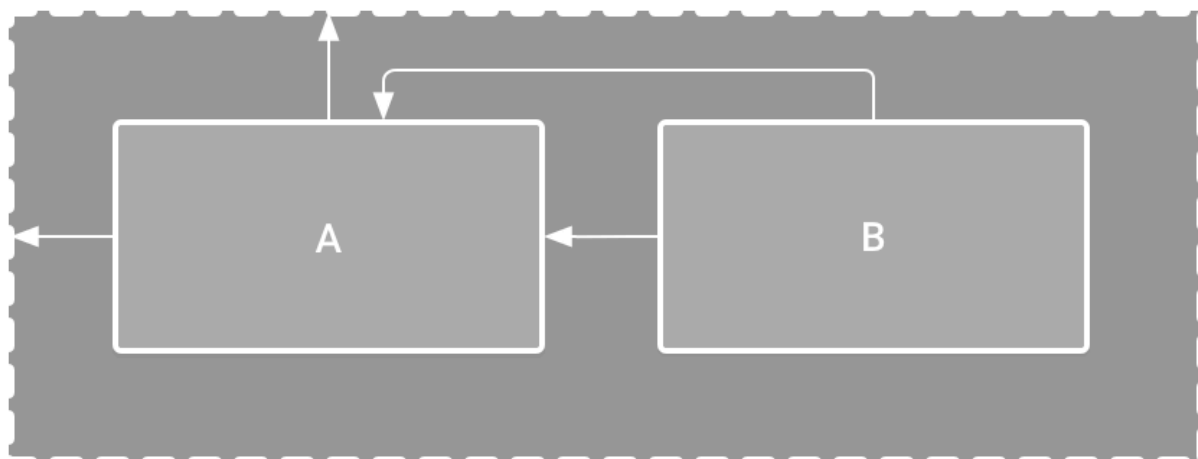


**Figura 6:** Visualización de `activity_main.xml` en el editor de diseño

Para obtener información adicional, consulta la [introducción al editor de diseño](#).

En el panel **Component Tree**, en la parte inferior izquierda, se muestra la jerarquía de vistas del diseño. En este caso, la vista de raíz es un elemento **ConstraintLayout**, que contiene solo un objeto **TextView**.

**ConstraintLayout** es un diseño que define la posición de cada vista a partir de restricciones sobre el diseño de nivel superior y las vistas secundarias. De esta manera, puedes crear diseños simples y complejos con una jerarquía de vistas plana. Este tipo de diseño evita la necesidad de diseños anidados. Un diseño anidado (es decir, un diseño dentro de otro), como se muestra en la figura 5, puede aumentar el tiempo necesario para dibujar la IU.



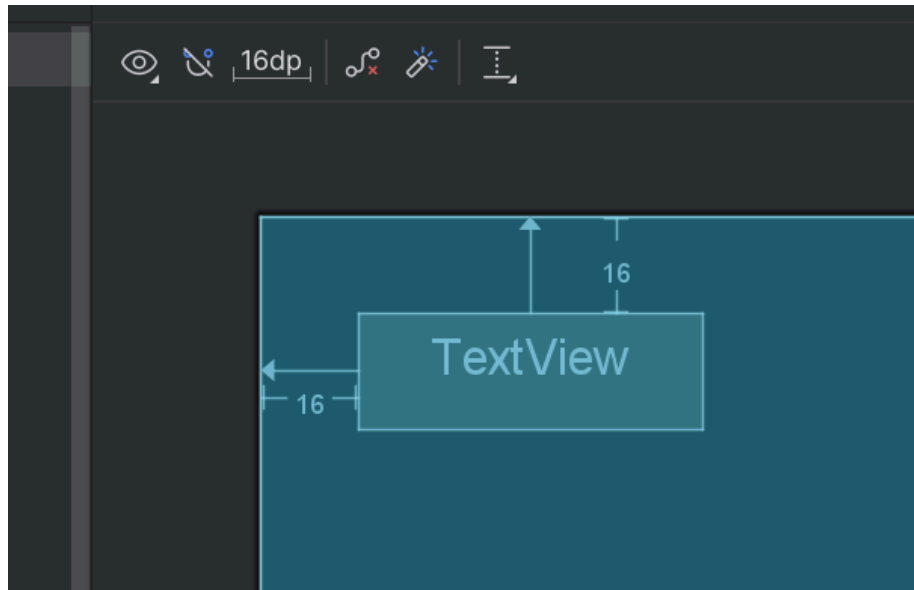
**Figura 7:** Ilustración de dos vistas ubicadas dentro de **ConstraintLayout**.

Por ejemplo, puedes declarar el siguiente diseño, que se muestra en la figura 7:

- La vista A se muestra a 16 dp de la parte superior del diseño de nivel superior.
- La vista A se muestra a 16 dp de la parte izquierda del diseño de nivel superior.
- La vista B se muestra a 16 dp a la derecha de la vista A.
- La vista B se encuentra alineada con la parte superior de la vista A.

En las siguientes secciones, compilarás un diseño similar al de la figura 4.

## Cómo agregar un cuadro de texto



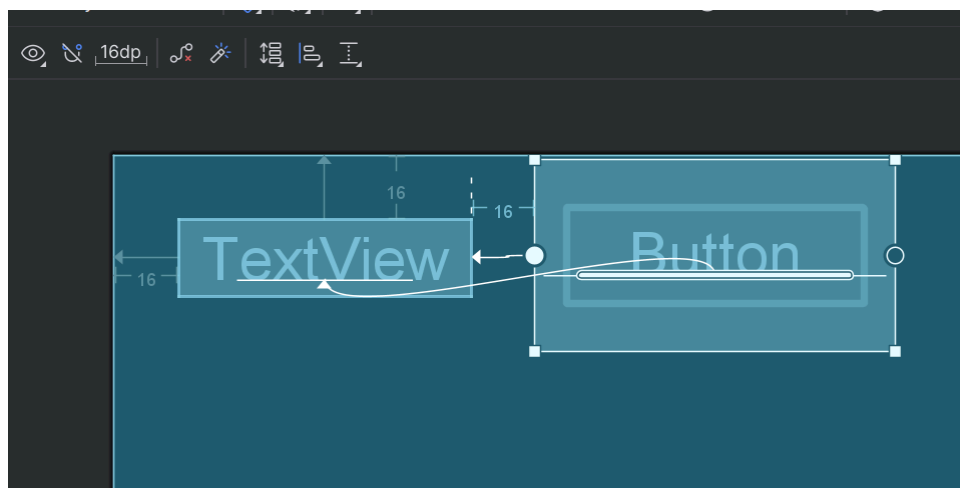
**Figura 8:** El cuadro de texto tiene restricciones en las partes izquierda y superior del diseño de nivel superior.

Sigue estos pasos para agregar un cuadro de texto:

1. Primero debes quitar todo lo que hay en el diseño. Haz clic en **TextView**, en el panel **Component Tree**, y presiona la tecla Delete.
2. En el panel **Palette**, haz clic en **Text** para mostrar los controles de texto disponibles.
3. Arrastra la opción **Plain Text** al editor de diseño y suéltala cerca de la parte superior del diseño. Este es un widget [EditText](#) que acepta entradas de texto sin formato.
4. Haz clic en la vista del editor de diseño. Ahora puedes ver los controladores cuadrados para cambiar el tamaño de la vista en cada esquina y los anclajes de restricciones circulares en cada lado. Si deseas obtener un mejor control, quizás te convenga acercar el editor. Para ello, usa los botones de **Zoom** en la barra de herramientas del editor de diseño.
5. Mantén presionado el anclaje en la parte superior, arrástralo hasta que se ajuste a la parte superior del diseño y luego suéltalo. De esta manera, se limita la vista dentro del margen predeterminado que se definió. En este caso, estableciste el valor en 16 dp desde la parte superior del diseño.
6. Usa el mismo proceso para crear una restricción desde el lado izquierdo de la vista hasta el lado izquierdo del diseño.

El resultado debería verse como en la figura 8.

## Cómo agregar un botón



**Figura 9:** El botón tiene restricciones respecto del lado derecho del cuadro de texto y de su línea de base.

1. En el panel **Palette**, haz clic en **Buttons**.
2. Arrastra el widget **Button** al editor de diseño y suéltalo cerca del lado derecho.
3. Crea una restricción desde el lado izquierdo del botón hasta el lado derecho del cuadro de texto.
4. Para restringir las vistas en una alineación horizontal, crea una restricción entre las líneas de base del texto. Para ello, haz clic con el botón derecho y, luego, selecciona

### Show Baseline



. El anclaje de línea de base aparece dentro del botón. Mantén presionado este anclaje y arrástralo hacia el anclaje de la línea de base que se muestra en el cuadro de texto adyacente.

El resultado debería verse como en la figura 9.

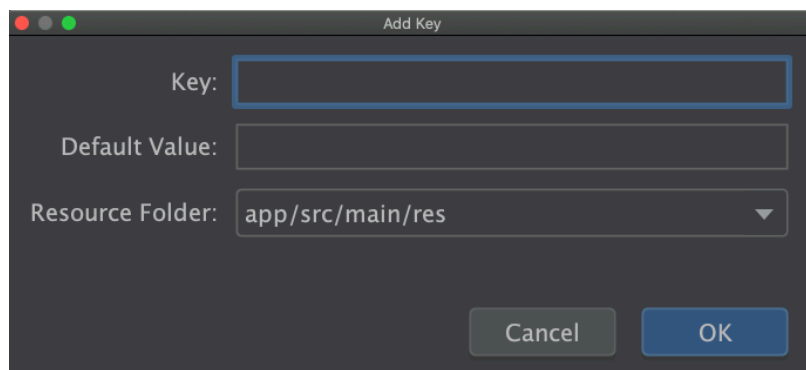
## Cómo cambiar las strings de la IU

Para obtener una vista previa de la IU, haz clic en  (**Select Design Surface**), en la barra de herramientas, y selecciona **Design**. Observa que la entrada de texto y la etiqueta del botón están configuradas con los valores predeterminados.

Sigue estos pasos para cambiar las strings de la IU:

1. Abre la ventana **Project** y, luego, abre **app > res > values > strings.xml**.  
Este es un archivo de [recursos de strings](#) en el que puedes especificar todas tus strings de IU. De esta manera, puedes administrar todas tus strings de IU desde una sola ubicación para que sea más fácil encontrarlas, actualizarlas y localizarlas.
2. Haz clic en **Open editor** en la parte superior de la ventana. Se abrirá el [editor de traducciones](#), que proporciona una interfaz simple para agregar y editar tus strings personalizadas. También te ayuda a mantener organizadas todas tus strings traducidas.

3. Haz clic en  (**Add Key**) para crear una string nueva como "texto de sugerencia" para el cuadro de texto. Se abrirá la ventana que puedes ver en la figura 7.



**Figura 10:** Diálogo para agregar una string nueva

En el cuadro de diálogo **Add Key**, completa los siguientes pasos:

1. Escribe "edit\_message" en el campo **Key**.
2. Introduce "Enter a message" en el campo **Default Value**.
3. Haz clic en **OK**.
4. Agrega otra clave llamada "button\_send" con un valor de "Send".

Ahora podrás configurar estas strings para cada vista. Para volver al archivo de diseño, haz clic en **activity\_main.xml** en la barra de pestañas. Luego, agrega las strings de la siguiente manera:

1. Haz clic en el cuadro de texto del diseño. Si la ventana **Attributes** no está visible del lado derecho, haz clic en **Attributes** en la barra lateral de la derecha.

2. Busca la propiedad **text** y, luego, haz clic en  (**Pick a Resource**), que se encuentra a la derecha del cuadro de texto. En el diálogo que aparece, haz doble clic en la opción **edit\_message** de la lista.

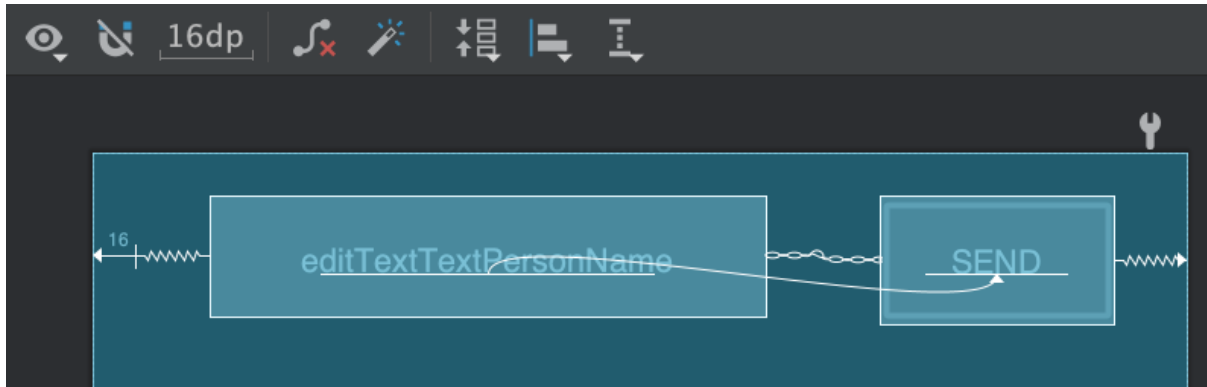
3. Haz clic en el botón del diseño y ubica su propiedad **text**, que actualmente está definida como "Button". Luego, haz clic en  (**Pick a Resource**) y selecciona **button\_send**.

## Cómo configurar un tamaño flexible para el cuadro de texto

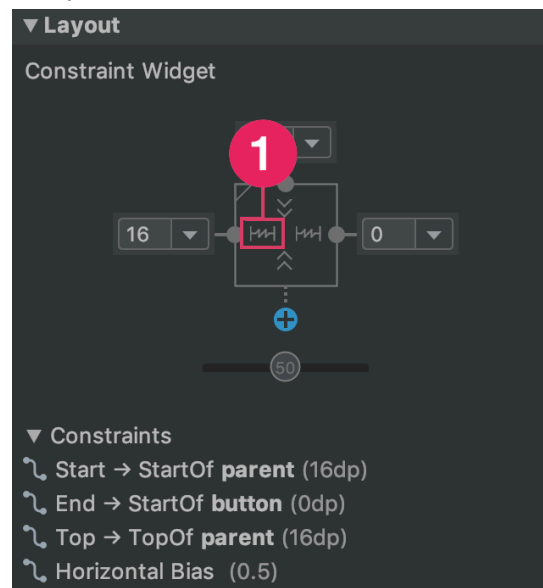
Para crear un diseño que reaccione a distintos tamaños de pantalla, debes hacer que el cuadro de texto se extienda para rellenar todo el espacio horizontal que queda una vez que se tienen en cuenta el botón y los márgenes.

Antes de continuar, haz clic en  (**Select Design Surface**), en la barra de herramientas, y selecciona **Blueprint**.

Para que el cuadro de texto sea flexible, sigue estos pasos:



**Figura 11:** El resultado que se obtiene cuando seleccionas **Create Horizontal Chain**.



**Figura 12:** Haz clic para cambiar el ancho a fin de **ajustar a las restricciones**.



**Figura 13:** El cuadro de texto se estirará hasta llenar el espacio sobrante.

1. Selecciona ambas vistas. Para ello, haz clic en una, mantén presionada la tecla Mayúsculas y, luego, haz clic en la otra vista. A continuación, haz clic con el botón

derecho en cualquiera de ellas y selecciona **Chains > Create Horizontal Chain**. Luego, se muestra un diseño como el de la figura 11.

Una [cadena \(chain\)](#) es una restricción bidireccional entre dos o varias vistas que te permite organizar las vistas encadenadas de forma simultánea.

2. Selecciona el botón y abre la ventana **Attributes**. Luego, usa **Constraint Widget** para establecer el margen derecho en 16 dp.
3. Haz clic en el cuadro de texto para ver sus atributos. Luego, haz clic en el indicador de ancho dos veces para establecerlo en una línea angulada (**Match Constraints**), como se indica en el texto destacado 1 de la figura 12.

*Match Constraints* significa que el ancho se expande para cumplir con la definición de las limitaciones horizontales y los márgenes. Por lo tanto, el cuadro de texto se extiende hasta llenar el espacio horizontal que queda después del botón y se registran todos los márgenes.

El diseño quedará listo y tendrá el aspecto de la figura 13.

Si te parece que tu diseño no quedó como esperabas, haz clic en **Observa el XML del diseño final** a continuación y accederás a una vista de cómo debería verse el XML. Compáralo con lo que ves en la pestaña **Code**. No te preocupes si tus atributos aparecen en un orden diferente; no es un error.

Para obtener más información sobre las cadenas y todo lo que puedes hacer con **ConstraintLayout**, lee [Cómo crear una IU responsiva con ConstraintLayout](#).

## 2. Ejecutar la app en un dispositivo real y en un emulador

Sigue este tutorial para ver cómo crear dispositivos virtuales y también a ejecutar en tu dispositivo las aplicaciones creadas.

Los objetivos son:

- Ejecutar la app anterior en tu dispositivo personal si tienes un dispositivo Android.
- Ejecutar la app en dos dispositivos virtuales (AVD - Android Virtual Devices) diferentes con pantallas de tamaño distinto (por ejemplo: "Pixel XL" y "Nexus S").

### Cómo ejecutar la app en un dispositivo real

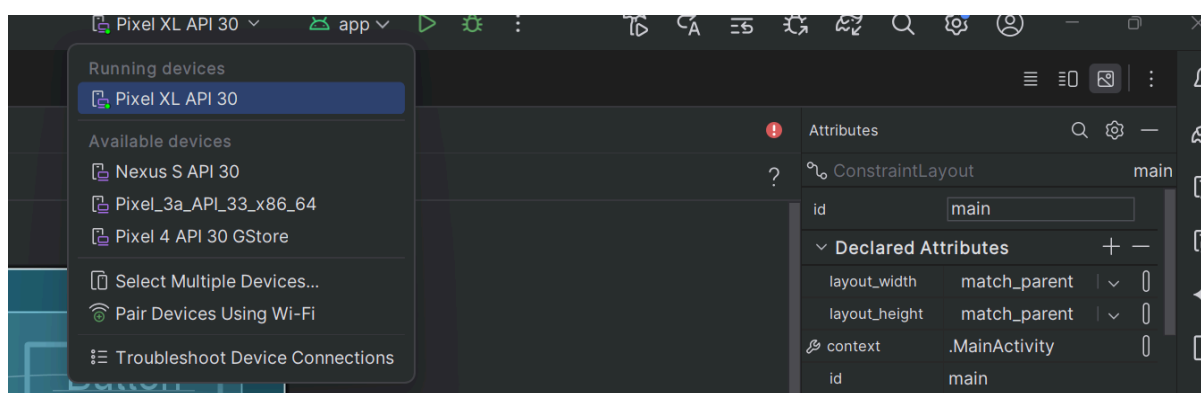
Configura tu dispositivo de la siguiente manera:

1. Conecta el dispositivo a la máquina de desarrollo con un cable USB. Si desarrollaste una app en Windows, es posible que debas [instalar el controlador USB adecuado](#) para tu dispositivo.
2. Completa los siguientes pasos a fin de habilitar la **depuración por USB** en la ventana **Opciones para desarrolladores**:
  1. Abre la app de **Configuración**.
  2. Si tu dispositivo usa Android R (11.0) o una versión posterior, selecciona **Sistema**. De lo contrario, continúa con el paso siguiente.

3. Desplázate hasta la parte inferior y selecciona **Acerca del teléfono**.
4. Desplázate hasta la parte inferior y presiona **Número de compilación** siete veces.
5. Regresa a la pantalla anterior, desplázate hasta la parte inferior y presiona **Opciones para desarrolladores**.
6. En la ventana **Opciones para desarrolladores**, desplázate hacia abajo para buscar y habilitar la **depuración por USB**.

Ejecuta la app en tu dispositivo de la siguiente manera:

1. En Android Studio, selecciona tu app desde el menú desplegable de configuraciones de ejecución y depuración de la barra de herramientas.
2. En la barra de herramientas, selecciona el dispositivo en el que deseas ejecutar la app desde el menú desplegable del dispositivo de destino.



**Figura 14:** Menú desplegable del dispositivo de destino.

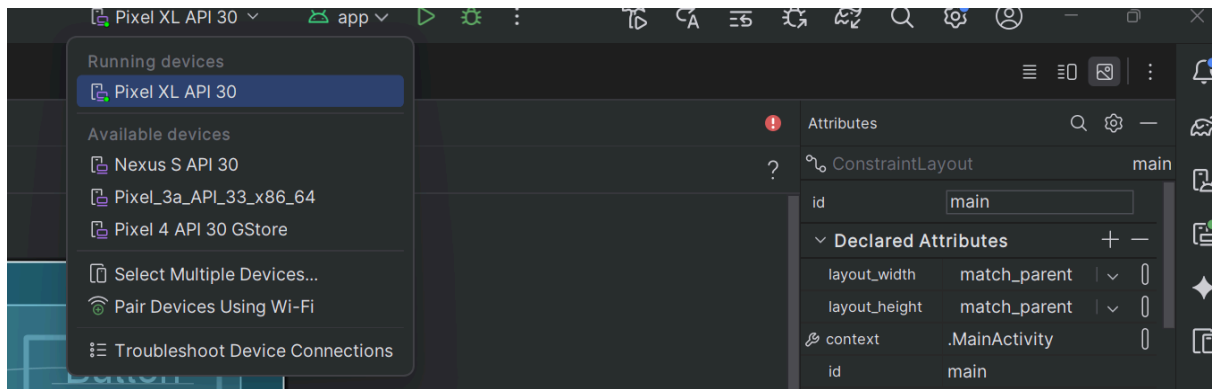
3. Haz clic en **Run** .

Android Studio instala tu app en el dispositivo conectado y la inicia. Ahora, aparecerá el mensaje "Hello, World!" en la app.

## Cómo ejecutar la app en un emulador

Ejecuta la app en un emulador de la siguiente manera:

1. En Android Studio, [crea un dispositivo virtual de Android \(AVD\)](#) que el emulador pueda usar para instalar y ejecutar la app.
2. En la barra de herramientas, selecciona tu app desde el menú desplegable de configuraciones de ejecución y depuración.
3. En el menú desplegable del dispositivo de destino, selecciona el AVD en el que deseas ejecutar la app.



**Figura 15:** Menú desplegable del dispositivo de destino

4. Haz clic en **Run** .

Android Studio instala la app en el AVD y, luego, inicia el emulador. Ahora, aparecerá el mensaje "Hello, World!" en la app.

### 3. Funcionalidad del botón y segunda vista.

**Nota:** Aunque en esta práctica aparecen los Intents, no los vamos a necesitar en la práctica 1 ya que tendremos una única Actividad. Sin embargo, más adelante, veremos cómo los Intents nos sirven para comunicarnos también con aplicaciones diferentes a la que estamos desarrollando (por ejemplo X -Twitter) o a abrir nuestro juego desde una notificación push.

#### Cómo responder al botón Enviar

Sigue estos pasos para agregar un método a la clase `MainActivity`, a la que se llama cuando presionas el botón **Enviar**:

En el archivo `app > java > com.example.myfirstapp > MainActivity`, agrega el siguiente método `sendMessage()`:

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    /** Called when the user taps the Send button */
    public void sendMessage(View view) {
        // Do something in response to button
    }
}
```

1. Es posible que veas un error porque Android Studio no puede resolver la clase `View` que se usa como argumento del método. Para borrar el error, haz clic en la declaración `View`, coloca el cursor sobre ella y presiona Alt + Intro (o, en Mac, Opción + Intro) a fin de realizar una corrección rápida. Si aparece un menú, selecciona **Import class**.
2. Regresa al archivo `activity_main.xml` para llamar al método desde el botón, de la siguiente manera:
  - Selecciona el botón en el editor de diseño.
  - En la ventana **Attributes**, ubica la propiedad **onClick** y selecciona **sendMessage [MainActivity]** de la lista desplegable.
3. De esta manera, cuando se presione el botón, el sistema llamará al método `sendMessage()`.

Toma nota de los detalles de este método. Son necesarios para que el sistema reconozca el método como compatible con el atributo `android:onClick`. Específicamente, el método tiene las siguientes características:

- Acceso público
  - Un valor que se muestra de void
  - Un objeto `View` como único parámetro, que es el objeto `View` en el que hiciste clic al final del paso 1
4. A continuación, deberás completar este método para leer el contenido del campo de texto y proporcionar dicho texto a otra actividad

## Cómo compilar un intent

Un `Intent` es un objeto que proporciona vinculación en tiempo de ejecución entre componentes separados, como dos actividades. El `Intent` representa la intención que tiene una app de realizar una tarea. Puedes usar intents para varias tareas; pero, en esta lección, tu intent inicia otra actividad.

En `MainActivity`, agrega la constante `EXTRA_MESSAGE` y el código `sendMessage()`, como se muestra a continuación:

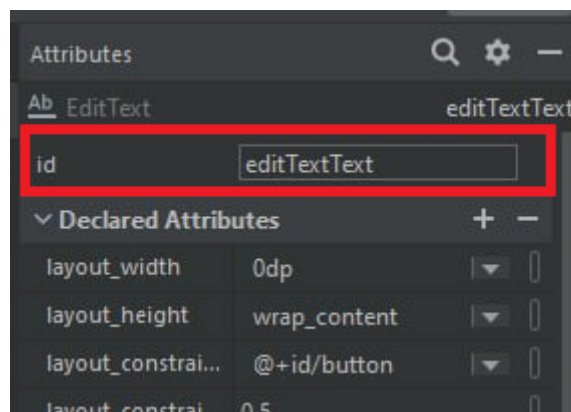
```

public class MainActivity extends AppCompatActivity {
    public static final String EXTRA_MESSAGE = "com.example.myfirstapp.MESSAGE";
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    /** Called when the user taps the Send button */
    public void sendMessage(View view) {
        Intent intent = new Intent(this, DisplayMessageActivity.class);
        EditText editText = (EditText) findViewById(R.id.editTextTextPersonName);
        String message = editText.getText().toString();
        intent.putExtra(EXTRA_MESSAGE, message);
        startActivity(intent);
    }
}

```

`R.id.editTextTextPersonName` es el ID asignado al elemento `TextView`, en tu caso puedes haber asignado un ID diferente (Figura 16).



**Figura 16.** ID de un elemento View

Todavía queda un error para `DisplayMessageActivity`, pero no te preocupes porque lo solucionarás en el siguiente paso.

Esto es lo que sucede en `sendMessage()`:

- El constructor `Intent` toma dos parámetros, un `Context` y una `Class`. El parámetro `Context` se usa primero porque la clase `Activity` es una subclase de `Context`. El parámetro `Class` del componente de la app, al que el sistema entrega el `Intent`, es, en este caso, la actividad que va a comenzar.
- El método `putExtra()` agrega el valor de `EditText` al intent. Un `Intent` puede transportar tipos de datos como pares clave-valor llamados *extras*. Tu clave es una constante pública `EXTRA_MESSAGE` porque la actividad siguiente

usa la clave para obtener el valor de texto. Te recomendamos definir las claves para extras de intents con el nombre del paquete de la app como prefijo. De esta manera, se garantiza que las claves sean únicas en el caso de que tu app interactúe con otras aplicaciones.

- El método `startActivity()` inicia una instancia de `DisplayMessageActivity` que especifica el `Intent`. Luego, deberás crear esa clase.

## Cómo crear la segunda actividad

Para crear la segunda actividad, sigue estos pasos:

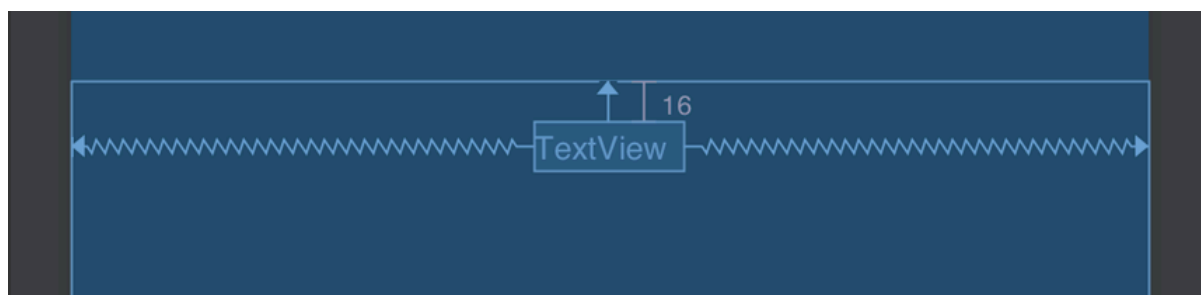
1. En la ventana **Project**, haz clic con el botón derecho en la carpeta de la **app** y selecciona **New > Activity > Empty Views Activity**.
2. En la ventana **Configure Activity**, ingresa "DisplayMessageActivity" en **Activity Name**. Deja todas las demás propiedades con sus valores predeterminados y haz clic en **Finish**.

Android Studio realiza tres acciones automáticamente:

- Crea el archivo `DisplayMessageActivity`.
- Crea el archivo de diseño `activity_display_message.xml`, que se corresponde con el archivo `DisplayMessageActivity`.
- Agrega el elemento `<activity>` obligatorio en `AndroidManifest.xml`.

Si ejecutas la app y presionas el botón en la primera actividad, se iniciará la segunda. Sin embargo, estará vacía porque usa el diseño vacío proporcionado por la plantilla.

## Cómo agregar una vista de texto



**Figura 17:** Vista de texto centrada en la parte superior del diseño.

La nueva actividad incluye un archivo de diseño en blanco. Sigue estos pasos para agregar una vista de texto al lugar donde aparece el mensaje:

1. Abre el archivo `app > res > layout > activity_display_message.xml`.
2. Haz clic en **Enable Autoconnection to Parent**  en la barra de herramientas. De esta manera, se habilita la conexión automática. Consulta la figura 1.

3. En el panel **Palette**, haz clic en **Text**, arrastra un elemento **TextView** al diseño y suéltalo cerca de la parte central superior del diseño de manera que se ajuste a la línea vertical que aparecerá. La opción de conexión automática agrega restricciones a la derecha y a la izquierda para ubicar la vista en el centro horizontal.
4. Crea una restricción más desde la parte superior de la vista de texto hasta la parte superior del diseño para que se vea como en la figura 17.

De manera opcional, puedes realizar algunos ajustes en el estilo de texto si expandes **textAppearance** en el panel **Common Attributes** de la ventana **Attributes** y cambias los atributos como **textSize** y **textColor**.

## Cómo mostrar el mensaje

En este paso, puedes modificar la segunda actividad para mostrar el mensaje que pasó la primera actividad.

En `DisplayMessageActivity`, agrega el siguiente código al método `onCreate()`:

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_display_message);

    // Get the Intent that started this activity and extract the string
    Intent intent = getIntent();
    String message = intent.getStringExtra(MainActivity.EXTRA_MESSAGE);

    // Capture the layout's TextView and set the string as its text
    TextView textView = findViewById(R.id.textView);
    textView.setText(message);
}
```

## 4. Entendiendo el ciclo de vida de una aplicación en Android

Aprovecha que ya sabes como ejecutar tu aplicación en Android Studio para entender los eventos que forman parte del [ciclo de vida de una aplicación Android](#) y familiarizarte con el Logcat.

Sobrescribe los métodos `onStart()`, `onRestart()`, `onResume()`, `onPause()`, `onStop()`, `onDestroy()` de tus actividades; pon puntos de interrupción; y escribe [mensajes de registro en estos métodos](#). Luego ejecuta la aplicación en el AVD y comprueba el Logcat.

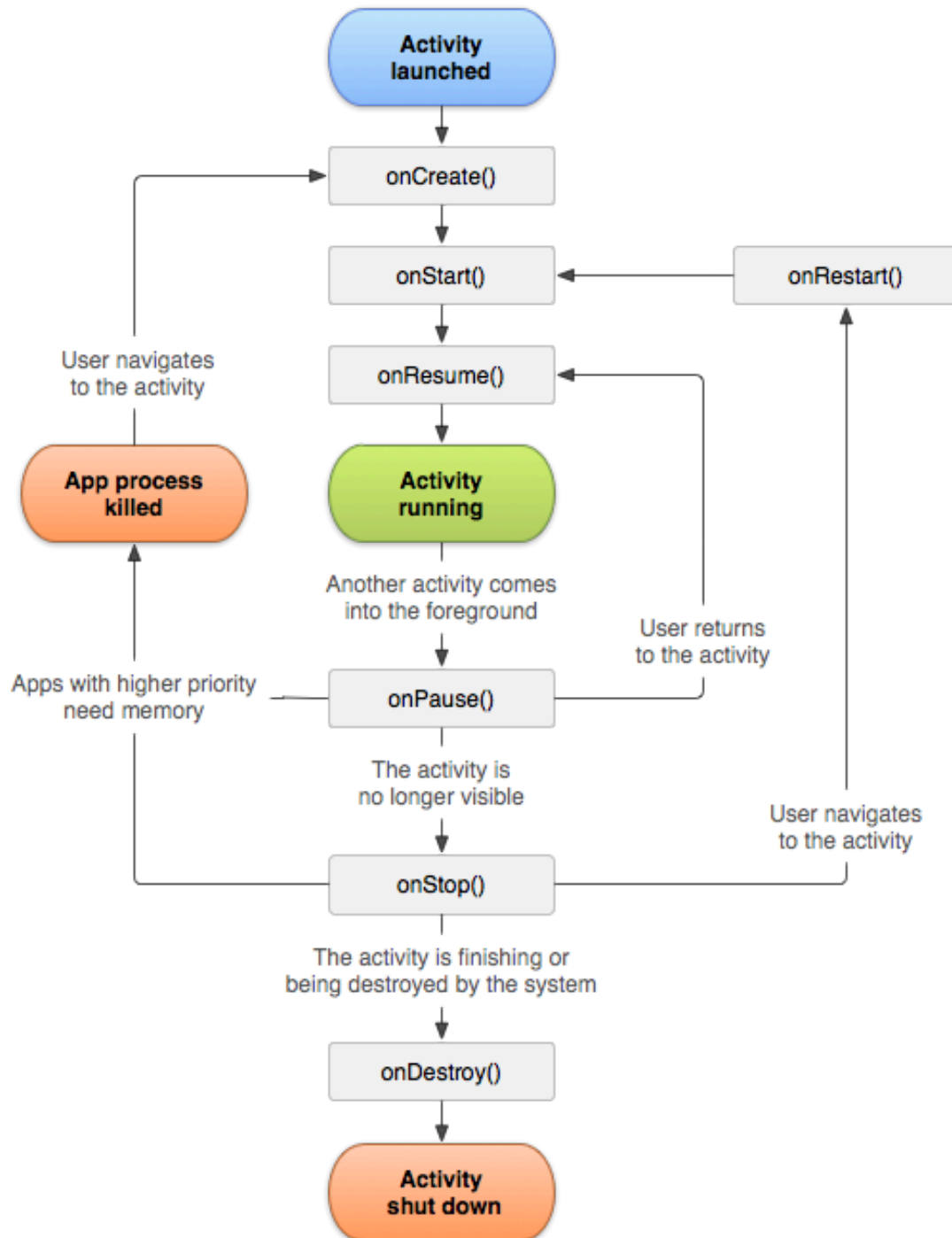
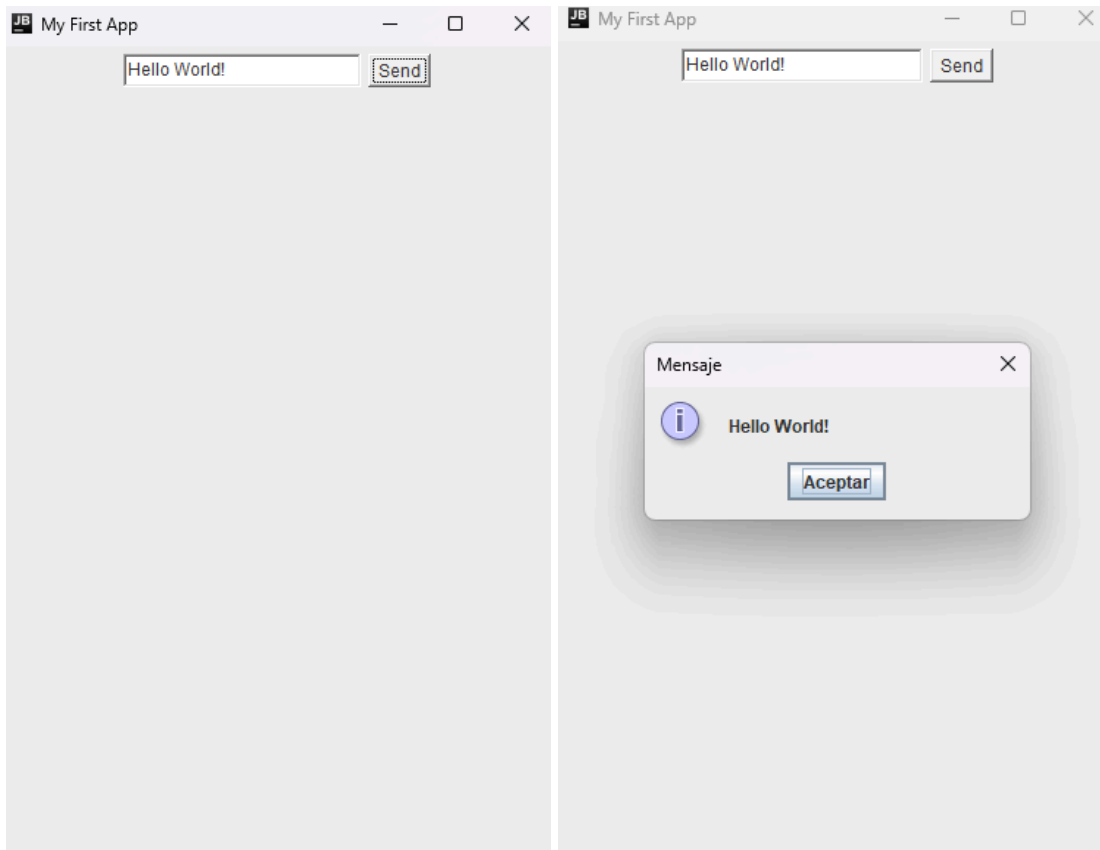


Fig. 18: Ciclo de vida de Android

#### 4. Un ¡Hola Mundo! en Escritorio (Swing)

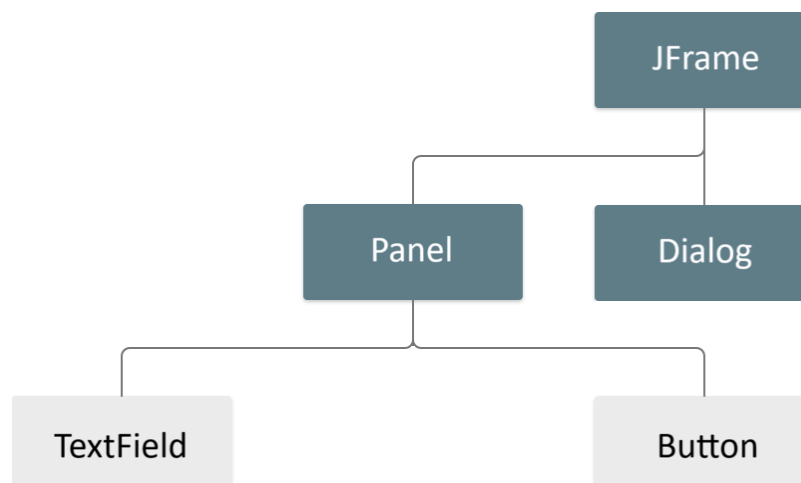
En este caso vamos a crear una aplicación similar a la de Android. Pero trabajaremos en el módulo que hemos creado al principio de tipo *“Java or Kotlin Library”*.

El objetivo ahora será crear una aplicación como la siguiente:



**Fig. 19:** Objetivo aplicación escritorio

Al igual que en Android, tendremos una jerarquía de vistas. En este caso:



**Fig. 20:** Jerarquía de componentes AWT

Vamos a necesitar las siguientes Clases de las librerías `java.awt`:

- [java.swing.JFrame](#) (Crea nuestra ventana)
- [java.awt.Button](#) (Componente que representa un botón)
- [java.awt.TextField](#) (Componente que representa texto)
- [java.awt.event.MouseListener](#) (Nos permite escuchar los eventos de ratón)

El punto de entrada a la ejecución de nuestra aplicación de Desktop es el método estático main. Si en la clase que contiene el módulo desktop escribimos main y pulsamos enter, el IDE debería autocompletar con lo necesario:

```
public static void main(String[] args) {  
  
}
```

## 1. Crear la ventana gráfica

En este caso trabajaremos por código, dentro de la función main de la clase principal de nuestro módulo de Java. Lo primero que necesitamos es crear la ventana que contendrá todo. En este caso vamos a instanciar un JFrame dentro de nuestro método Main.

Una vez instanciado llamaremos a las siguientes funciones (revisa los enlaces a la documentación oficial para familiarizarte) para darle un tamaño a la ventana y hacerla visible y para hacer que la aplicación se detenga si cerramos la ventana:

[.setSize\(w, h\)](#)  
[.setDefaultCloseOperation\(\)](#)  
[.setVisible\(true\)](#)

```
public static void main(String[] args) {  
    JFrame myScreen = new JFrame();  
    myScreen.setSize(400, 600);  
  
    myScreen.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);  
    myScreen.setVisible(true);  
}
```

## 2. Añadir el campo de texto y el botón

Necesitamos darle un Layout a nuestro JFrame, por ejemplo [FlowLayout](#).

Lo siguiente será instanciar nuestro [TextField](#) y nuestro [Button](#) e introducirlos dentro del objeto JFrame mediante el método que tienen los contenedores como Panel, [.add\(\)](#).

Mirad la documentación para ver cómo podemos añadir un layout y elementos en nuestro JFrame.

## 3. Funcionalidad del botón y la ventana de diálogo

Por último necesitamos añadir la funcionalidad al botón para que lance una ventana de diálogo con el texto introducido. Esta ventana emergente será un [JOptionPane](#).

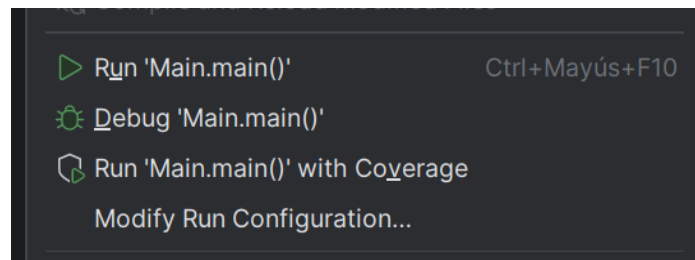
Para ello asignaremos el listener correspondiente ([MouseListener](#)) a nuestra instancia de Button con [.addMouseListener\(\)](#) para escuchar los [eventos de ratón](#). En `mouseReleased()` pondremos el siguiente código:

```
JOptionPane.showMessageDialog(window, text.getText());
```

//window es el JFrame y text será el texto que queremos que se muestre.

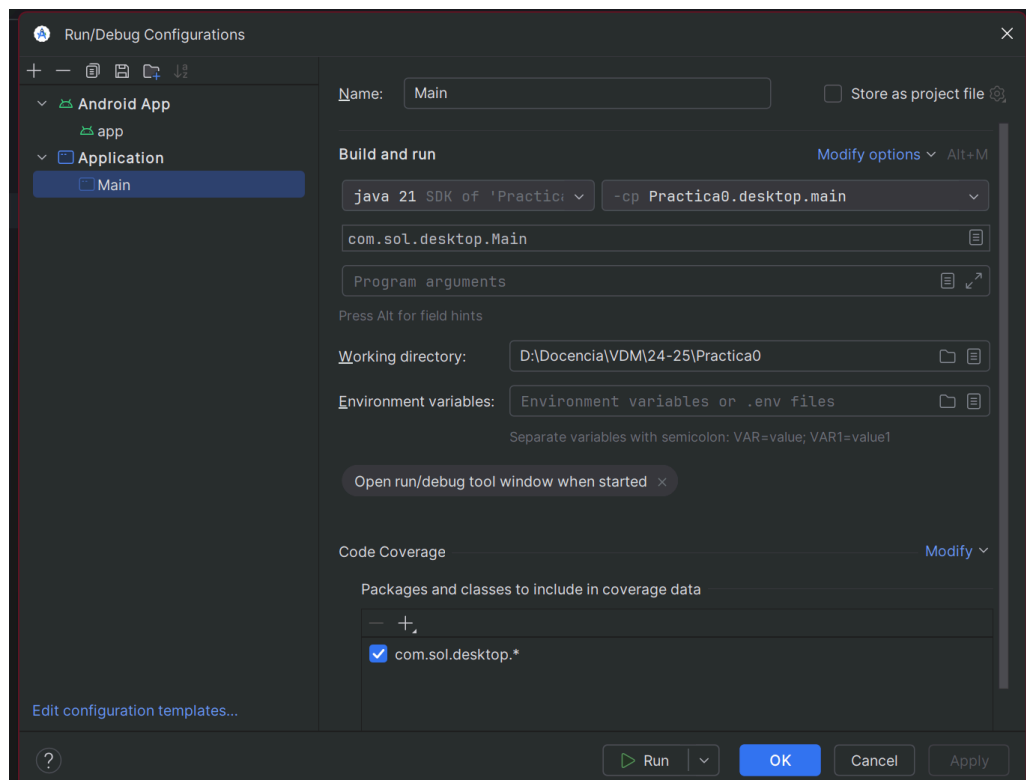
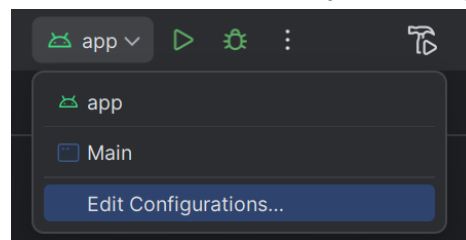
#### 4. Solo falta ejecutar

Para ejecutar nuestra aplicación de escritorio lo más rápido la primera vez es hacer clic derecho dentro del código de nuestra clase que contiene el método main() y pulsar “Run” en el menú contextual Fig. 21.



**Fig. 21:** Menú contextual

Tanto en Android como en Desktop tenemos también la opción de configurar la ejecución, pudiendo cambiar por ejemplo el JDK, dándole un nombre a la configuración de ejecución o cambiando el archivo .class que contiene el main a ejecutar (Fig. 22).



**Fig. 22:** Cambiar la configuración de la ejecución

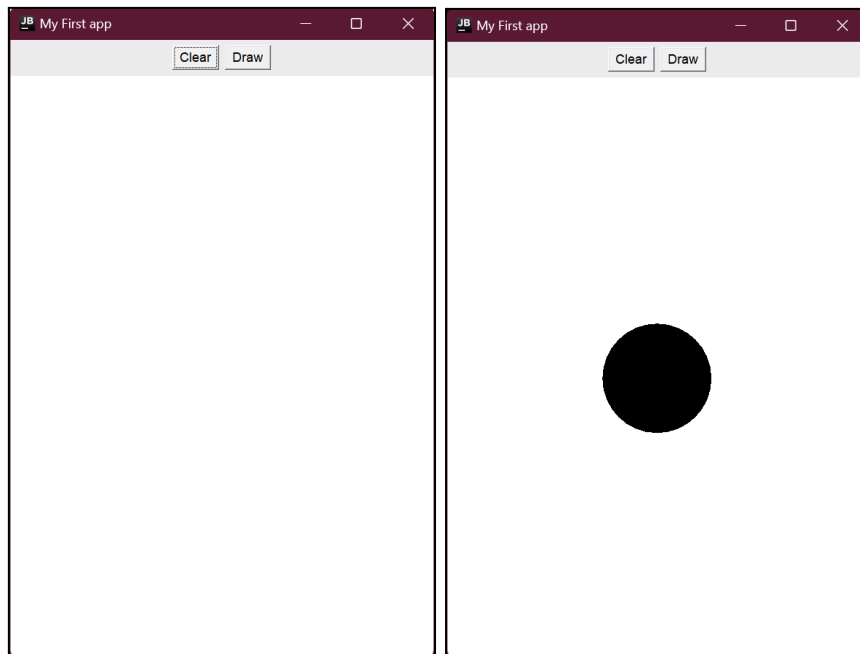
## 5. Sigüientes pasos, un poco más cerca de nuestro juego!

Lo que hemos visto hasta ahora son clases que nos ayuda a crear botones y texto como elementos de interfaz, sin embargo, a la hora de crear videojuegos lo que nos va a interesar es trabajar en un lienzo (Canvas) que nos permita el control sobre los pixeles que mostramos en pantalla, dibujando imágenes y figuras geométricas.

En Android vamos querer tener una vista del tipo [android.view.SurfaceView](#) en nuestra actividad y para escritorio tendremos un componente [java.awt.Canvas](#).

Así que ahora, vamos a añadir un Canvas en ambas aplicaciones junto a 2 nuevos botones que hagan los siguiente:

- Dibujar una figura
- “Limpiar” Canvas



**Fig. 24:** Aplicación de escritorio, el botón Clear deja todo el canvas en blanco mientras que el botón Draw dibuja un círculo negro dentro del canvas.

Vamos a necesitar las siguientes librerías:

- Para escritorio de momento necesitamos
  - `java.awt.Canvas`
  - `java.awt.Graphics2D`

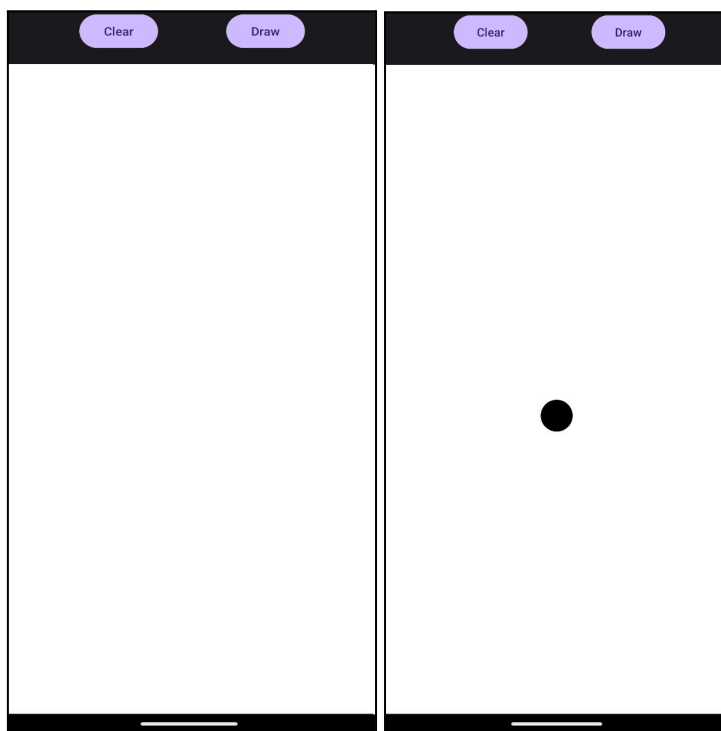
En escritorio, la clase Canvas nos proporciona un Graphics2D, clase que nos proporciona los métodos con los que pintar. Más adelante en la asignatura veremos más detalles de estas clases.

- Para Android necesitaremos :
  - `android.view.SurfaceView`
  - `android.view.SurfaceHolder`

- android.graphics.Canvas
- android.graphics.Paint

En android, podemos coger los elementos View que hemos definido con:  
`findViewById(R.id.idQueUsemosParaElElemento);`

Esta llamada recorre el árbol jerárquico de la vista para encontrar nuestro elemento deseado lo que puede tener una gran carga si lo hacemos repetidamente.



**Fig. 24:** Aplicación de Android, el botón Clear deja todo el canvas en blanco mientras que el botón Draw dibuja un círculo negro dentro del canvas.

Tienes algunas pistas en [\[1\]](#), [\[2\]](#), [\[3\]](#). Vamos a necesitar nuestro SurfaceView y los dos botones que afectarán a este cuando pulsemos. A los botones les añadiremos un OnClickListener en cuyo método onClick pintaremos en nuestro lienzo. En este caso para pintar, tenemos que pedir a nuestra vista SurfaceView el [SurfaceHolder](#), Interfaz de la superficie donde modificaremos los pixeles. En este caso la funcionalidad para pintar y modificar el mapa de bit que se muestra nos la da la clase [Canvas](#).

**Nota:** En la siguiente práctica tendremos únicamente nuestro lienzo y pintaremos nuestros elementos de juego e interfaz en este, por lo que será necesario crear y gestionar el input del jugador. Además, para modificar la superficie de pintado, trabajaremos en otro hilo. Esto es únicamente una primera aproximación a algunas de las clases que veremos.

## 6. Instrucciones de entrega

La práctica debe entregarse utilizando el mecanismo de entregas del campus virtual, no más tarde de la fecha y hora indicadas en el propio campus.

**La entrega consistirá en un archivo .zip con el proyecto completo de Android Studio.** Si el proyecto pesa demasiado, no os dejará subirlo. Aseguraros de limpiar los archivos temporales para que pese lo menos posible (carpeta **".gradle"** -y no gradle sin el punto- y cada una de las carpetas **"build"** dentro de cada uno de los módulos).

Se añadirá también un fichero **alumnos.txt** con el nombre completo de los alumnos que han trabajado juntos.

## **7. Evaluación de la práctica**

**Esta práctica no cuenta para nota.** Puede realizarse de manera **individual o en parejas**. No es necesario por tanto que los grupos sean los mismos con los que se realizarán el resto de prácticas que sí cuentan para la nota final de la asignatura.